Actividad Evaluativa 2 (AE2) Programación 1

Alumno: Kokotik Evaristo

Profesor: Ing. Joaquin Suarez

Institución Académica: Universidad de la Cuenca del Plata

Materia: Programación 1

CONSIGNA DE ACTIVIDAD DE EVALUACIÓN

Parte 1: Descripción del Problema y Justificación de Estructura de Datos

Descripción del problema.

El problema consiste en automatizar y persistir el registro de ventas del buffet de la facultad para reemplazar las anotaciones manuales. Los datos de entrada iniciales son el catálogo base de productos (código, nombre y precio) y el registro de cada transacción (día de la semana del 1 al 5, código del producto y cantidad vendida). El sistema procesa estos datos acumulando las cantidades vendidas y calculando los ingresos. Finalmente, la salida consta de reportes semanales detallados: unidades vendidas por producto, recaudación por día y total, producto más vendido y productos sin ventas.

Estructuras de Datos Seleccionados: Para resolver este caso de manera eficiente, se seleccionaron tres estructuras principales:

- Arreglo de diccionarios/registros: Para guardar el código, el nombre y el precio de cada producto, elegimos un arreglo de diccionarios (registros) en lugar de arreglos paralelos unidimensionales. Esta estructura es mucho más eficiente porque agrupa la información del producto de forma unificada bajo una misma entidad, eliminando el riesgo de que se desfasen los datos (por ejemplo, que el precio de un producto se mezcle con el nombre de otro).

- Arreglo bidimensional o Matriz: Para registrar las ventas de los 5 días hábiles, descartamos usar cinco vectores unidimensionales separados por día. En su lugar, utilizamos una matriz de 6 filas (productos) por 5 columnas (días de lunes a viernes). Esta elección es más adecuada porque permite utilizar bucles anidados para recorrer la información de forma ágil; leyendo una fila completa obtenemos rápidamente el total de unidades de un producto, y recorriendo una columna podemos calcular el total recaudado en un día específico cruzando los datos con el catálogo.

- Archivos de texto de organización secuencial: Dado que la memoria principal es volátil, si no guardamos la matriz de ventas en un archivo, toda la carga del día se perdería al cerrar el programa. Utilizamos un archivo secuencial para registrar las transacciones físicas. De esta manera, cada vez que el programa arranca, lee el archivo para volver a cargar la matriz en la memoria con todo lo acumulado en la semana y poder continuar exactamente desde donde se dejó

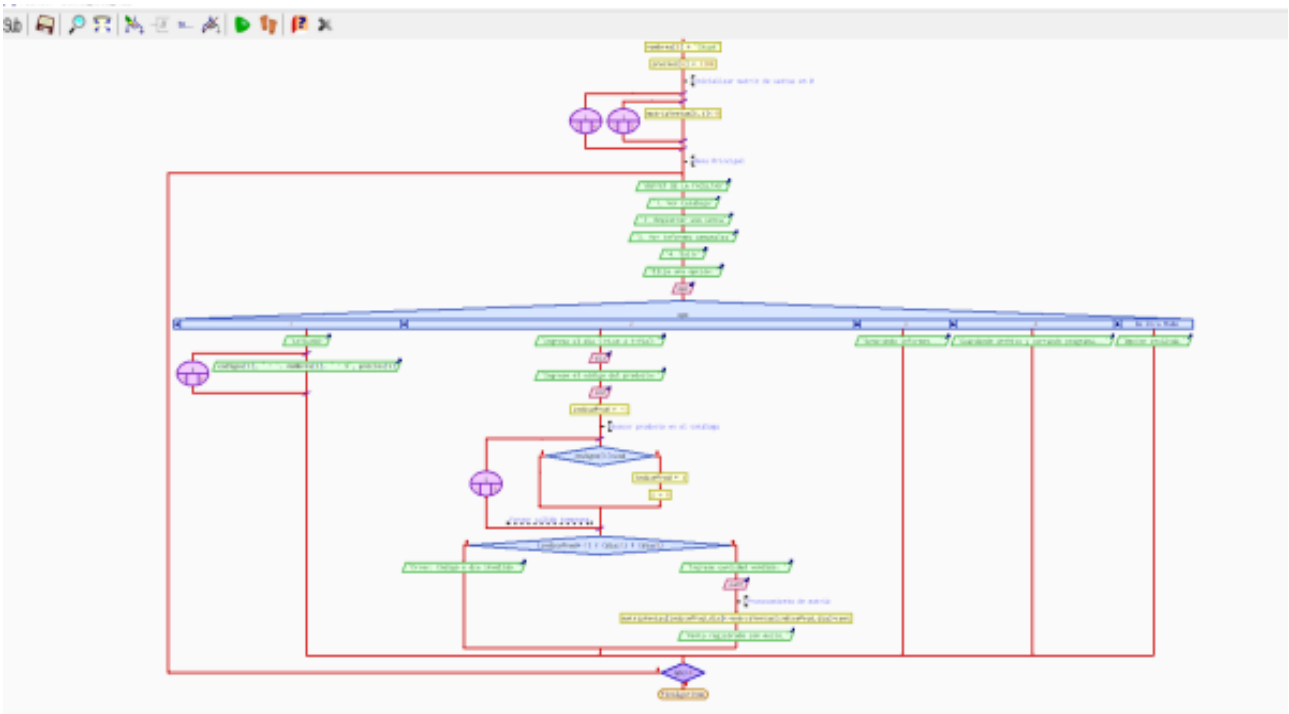
Parte 2: Diseño Lógico y Pseudocódigo (PSeInt)

Especificación del manejo de datos

- Carga inicial: Al iniciar el algoritmo, se definen arreglos paralelos para almacenar estáticamente el catálogo (códigos, nombres y precios) y se dimensiona una matriz de 6 filas por 5 columnas para las ventas de la semana, inicializándola en ceros.

- Procesamiento: El sistema se mantiene activo mediante un menú con la estructura Repetir - Hasta Que. Al registrar una venta, el algoritmo recibe el código del producto y el día. Busca secuencialmente el índice de ese código en el catálogo y utiliza ese índice (fila) junto con el día (columna) para ubicar la celda exacta y acumular la cantidad vendida: matriz_ventas[indicePord, dia] = matriz_ventas[indicePrud, dia] + cantidad.

- Recuperación: Cuando el usuario solicita los informes, el programa recupera la información iterando la matriz. Recorriendo una fila completa obtiene el total de unidades vendidas de un producto; y recorriendo una columna (multiplicando cada celda por el precio correspondiente en el catálogo), calcula la recaudación diaria total.



Parte 3: Codificación en Python

Primer avance del código: carga del catálogo del buffet

```python
# 1. Arreglo de Diccionarios para el catálogo
catalogo = [
    {"codigo": 101, "nombre": "Café con leche", "precio": 1500},
    {"codigo": 102, "nombre": "Medialuna", "precio": 700},
    {"codigo": 103, "nombre": "Sándwich de miga", "precio": 1800},
    {"codigo": 104, "nombre": "Agua mineral", "precio": 1000},
    {"codigo": 105, "nombre": "Empanada", "precio": 1200},
    {"codigo": 106, "nombre": "Chipá", "precio": 1300}
]

def cargar_ventas(matriz):
    #Creamos el archivo de ventas si no existe, para evitar errores al leerlo
    archivo_crear = open("ventas_facultad.txt", "a")
    archivo_crear.close()
```

Parte 4: Pruebas de Escritorio

Prueba 1 (Caso normal/Dato duplicado - Acumulación):

Dato de entrada: Registrar 5 unidades de "Café con leche" (Código 101) el lunes (Día 1). Luego, sin salir del programa, volver a registrar 3 unidades del mismo producto el mismo día.

Resultado esperado: La celda de la matriz debe acumular los valores; al pedir el informe de unidades vendidas, debe mostrar un total de 8 unidades de Café con leche, en lugar de sobrescribir el 5 original.

Resultado obtenido:

```
BUFFET DE LA FACULTAD
1. Ver Catálogo
2. Registrar una venta
3. Ver informes semanales
4. Salir
Elija una opción: 2
Ingrese el día (1=Lun a 5=Vie): 1
Ingrese el código del producto: 101
Ingrese la cantidad vendida: 5
Venta registrada con éxito.

BUFFET DE LA FACULTAD
1. Ver Catálogo
2. Registrar una venta
3. Ver informes semanales
4. Salir
Elija una opción: 2
Ingrese el día (1=Lun a 5=Vie): 1
Ingrese el código del producto: 101
Ingrese la cantidad vendida: 3
Venta registrada con éxito.

BUFFET DE LA FACULTAD
1. Ver Catálogo
2. Registrar una venta
3. Ver informes semanales
4. Salir
Elija una opción: 3

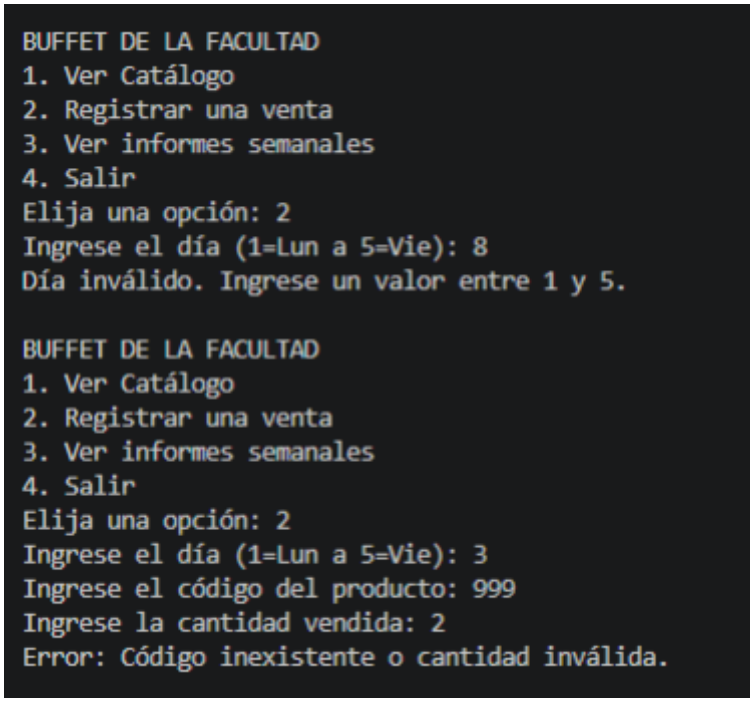
INFORMES DE LA SEMANA
a) Unidades vendidas por producto:
 - Café con leche: 8 unidades
```

Prueba 2 (Caso límite - Código fuera de rango):

Dato de entrada: Intentar registrar una venta ingresando un día inválido (ej. 8) o el código de producto 999 (inexistente).

Resultado esperado: El algoritmo debe detectar que los datos ingresados no superan la validación condicional y debe imprimir en consola el mensaje de error indicando que el código o el día es inválido, retornando al menú sin que el sistema colapse.

Respuesta Obtenida:



Prueba 3 (Caso límite - Archivo inexistente):

Dato de entrada: Iniciar el programa por primera vez habiendo eliminado previamente el archivo ventas_facultad.txt de la carpeta.

Resultado esperado: El sistema debe arrancar sin interrumpirse por errores de lectura, inicializar la matriz en ceros y mostrar el menú principal correctamente.

Resultado obtenido: El programa inició con éxito y creó el archivo en blanco al registrar la primera venta.

Parte 5: Reflexión sobre estructuras de datos

En la resolución de este caso, la principal alternativa de estructura de datos que descarté fue el uso de arreglos paralelos unidimensionales (por ejemplo, definir un vector distinto para cada uno de los cinco días de la semana). Tomé esta decisión porque manejar cinco vectores separados para representar un sistema de ventas unificado dificulta enormemente el mantenimiento del código, requiere escribir validaciones repetitivas y aumenta el riesgo de cometer errores lógicos al momento de cruzar la información para los informes semanales. La estructura que elegí finalmente fue una matriz bidimensional (6 filas por 5 columnas) para gestionar las transacciones dinámicas, combinada con un arreglo de diccionarios para mantener estático el catálogo. Lo que aprendí sobre las ventajas de esta elección es su alta modularidad: esta estructura permite recorrer la totalidad de la información y calcular los acumulados diarios o por producto utilizando bucles anidados de forma muy eficiente, reduciendo la cantidad de líneas de código.

CONSIGNAS DE ACTIVIDADES DE PROCESO: Portafolio Digital

1. Selección de estructuras:
Para resolver el caso del buffet de la facultad, decidí utilizar un arreglo de diccionarios para guardar el catálogo de productos porque me permite agrupar el código, nombre y precio sin que se desfasen. Para registrar las ventas de la semana, elegí utilizar una matriz bidimensional de 6 filas (por los productos) por 5 columnas (por los días), ya que esto me facilitará calcular los totales iterando con bucles anidados.

2. Borrador de diseño:

```
Algoritmo Buffet_facu_AE2
    // 1. Declaración de estructuras
    Definir opc, dia, cod, cant, i, j, indiceProd Como Entero
    Definir matrizVentas Como Entero
    Dimensionar matrizVentas(6,5)
    Definir codigos, precios Como Entero
    // Catálogo (Usamos arreglos)
    Definir nombres Como Cadena // Matriz 6 productos x 5 días
    Dimensionar codigos(6), nombres(6), precios(6)
    // Carga del catálogo
    codigos[1] ← 101
    nombres[1] ← 'Café con leche'
    precios[1] ← 1500
    codigos[2] ← 102
    nombres[2] ← 'Medialuna'
    precios[2] ← 700
    codigos[3] ← 103
    nombres[3] ← 'Sándwich de miga'
    precios[3] ← 1800
    codigos[4] ← 104
    nombres[4] ← 'Agua mineral'
    precios[4] ← 1000
    codigos[5] ← 105
    nombres[5] ← 'Empanada'
    precios[5] ← 1200
    codigos[6] ← 106
    nombres[6] ← 'Chipá'
    precios[6] ← 1300
    // Inicializar matriz de ventas en 0
    Para i←1 Hasta 6 Hacer
        Para j←1 Hasta 5 Hacer
            matrizVentas[i,j]←0
        FinPara
    FinPara
```

```
// Inicializar matriz de ventas en 0
Para i←1 Hasta 6 Hacer
    Para j←1 Hasta 5 Hacer
        matrizVentas[i,j]←0
    FinPara
FinPara
// Menú Principal
Repetir
    Escribir 'BUFFET DE LA FACULTAD'
    Escribir '1. Ver Catálogo'
    Escribir '2. Registrar una venta'
    Escribir '3. Ver informes semanales'
    Escribir '4. Salir'
    Escribir 'Elija una opción:'
    Leer opc
    Según opc Hacer
        1:
            Escribir 'CATÁLOGO'
            Para i←1 Hasta 6 Hacer
                Escribir codigos[i], ' - ', nombres[i], ' - $', precios[i]
            FinPara
```

3. Primer avance de código en Python

Python > Buffet_facu_AE2.py > ...

```python
# 1. Arreglo de Diccionarios para el catálogo
catalogo = [
    {"codigo": 101, "nombre": "Café con leche", "precio": 1500},
    {"codigo": 102, "nombre": "Medialuna", "precio": 700},
    {"codigo": 103, "nombre": "Sándwich de miga", "precio": 1800},
    {"codigo": 104, "nombre": "Agua mineral", "precio": 1000},
    {"codigo": 105, "nombre": "Empanada", "precio": 1200},
    {"codigo": 106, "nombre": "Chipá", "precio": 1300}
]

def cargar_ventas(matriz):
    #Creamos el archivo de ventas si no existe, para evitar errores al leerlo
    archivo_crear = open("ventas_facultad.txt", "a")
    archivo_crear.close()

    #Ahora que sabemos que existe, lo leemos sin problemas.
    archivo = open("ventas_facultad.txt", "r")
    for linea in archivo:
        datos = linea.strip().split(",")
        if len(datos) == 3:
            dia = int(datos[0])
            codigo = int(datos[1])
            cantidad = int(datos[2])

            #Buscar el índice del producto en el catálogo
            for i in range(len(catalogo)):
                if catalogo[i]["codigo"] == codigo:
                    matriz[i][dia - 1] = matriz[i][dia - 1] + cantidad
    archivo.close()

def guardar_venta(dia, codigo, cantidad):
    #Guardamos cada venta para no perder datos
    archivo = open("ventas_facultad.txt", "a")
    archivo.write(str(dia) + "," + str(codigo) + "," + str(cantidad) + "\n")
    archivo.close()
```

4. Registro de pruebas: Prueba inicial de funcionamiento de la matriz:

- **Dato de entrada / Acción:** Registrar manualmente 5 unidades de "Café con leche" (Código 101) para el lunes (Día 1) utilizando la opción 2 del menú interactivo.
- **Resultado esperado:** La primera celda de la matriz debe recibir el valor ingresado y al ejecutar los informes, debe mostrar 5 unidades vendidas de Café con leche.
- **Resultado obtenido:** El código funcionó correctamente y la consola imprimió "Venta registrada con éxito". Al pedir el informe, el sistema devolvió "Café con leche: 5 unidades", validando que la ubicación de fila y columna en la matriz es exacta